Predicting Electoral Outcomes in the 2019 Indian General Election Using Machine Learning

A Candidate-Level Classification Study Based on Election Affidavit and Constituency Data

Tejasvi Chaturvedi
Department of Computer Science, Ramanujan College, University of Delhi, India
College Roll No.: 24570082 | University Roll No.: 24020570066
Course: Machine Learning, B.Sc. (H) Computer Science, Semester VI
Supervisor: Mr. Nikhil Kumar | Problem No.: 49 | Original Research Article

Abstract. This study explores whether publicly reported candidate characteristics can help distinguish winning and non-winning candidates in the 2019 Indian General Election. The analysis uses the supplied LS_2.0.csv dataset, described in the project materials as candidate and election information. The original file contains 2,263 records and 19 columns. After removing 245 NOTA entries, 2,018 candidate records remained, comprising 539 winners and 1,479 non-winners. Predictors included party, state, declared assets and liabilities, criminal cases, age, education, gender and social category. Monetary variables were converted to numeric values and log-transformed. Missing values were handled using training-set imputation, categorical variables were one-hot encoded, numerical variables were standardized, and the minority class was oversampled in the training data only. Four classifiers were compared. On a stratified random hold-out set, Decision Tree achieved the highest F1 score (0.7751), while Logistic Regression recorded the highest ROC-AUC (0.8893). A separate constituency-held-out evaluation gave Logistic Regression the highest F1 score (0.7667) and ROC-AUC (0.9227). The results suggest that political and candidate-level features contain predictive information in this dataset. They do not establish causal relationships, and the findings should be interpreted in light of the dataset's scope and the limited number of validation splits.

Keywords: Indian General Election; candidate affidavits; election outcome; classification; Logistic Regression; Decision Tree; constituency validation

1. Introduction

Election outcomes are shaped by several factors, including party organization, local political conditions and candidate characteristics. In India, candidates contesting parliamentary elections submit affidavits that disclose information such as assets, liabilities and pending criminal cases. These records offer a useful starting point for studying electoral outcomes with data-analysis methods.

This project examines whether information available at the candidate level can be used to classify a candidate as a winner or a non-winner in the 2019 Indian General Election. Rather than treating any single attribute as the explanation for victory, the study compares several machine-learning models and checks whether their performance remains similar when entire constituencies are kept out of training.

The main objectives are to prepare the election dataset, explore the distribution of candidate attributes, compare four classification algorithms, and interpret the most influential features. The work is intended as a predictive study; it does not estimate the causal effect of wealth, criminal cases or party affiliation.

2. Dataset and Variables

The supplied file, LS_2.0.csv, contains 2,263 rows and 19 columns. The fields cover state, constituency, candidate name, winner status, party, election symbol, gender, criminal cases, age, category, education, assets, liabilities and vote-related information. The project notebook identifies the data source as Indian Candidates for General Election 2019. The dataset listing is hosted on Kaggle and is associated with candidate-affidavit and election-result information.

NOTA records were excluded because they do not represent individual candidates contesting the seat in the same way as the other rows. This removed 245 records and left 2,018 candidate records. The target variable was WINNER, coded as 1 for a winning candidate and 0 for a non-winning candidate. The final class distribution was 539 winners (26.71%) and 1,479 non-winners (73.29%).

The model used PARTY, STATE, ASSETS, LIABILITIES, CRIMINAL_CASES, AGE, EDUCATION, GENDER and CATEGORY as predictors. Vote totals and vote-share fields were not included in the feature set because they are directly connected to the election result and could make the prediction task misleading.

3. Data Cleaning and Preprocessing

Column names containing line breaks were standardized by replacing whitespace with underscores. The WINNER, AGE and CRIMINAL_CASES fields were converted to numeric values. Asset and liability values were originally stored as text containing currency labels and formatted amounts; the numeric amount was extracted after removing commas and other text.

Declared assets and liabilities were strongly right-skewed, so the notebook applied the log(1+x) transformation. In the final evaluation pipeline, numerical missing values were imputed using the median calculated from the training data. Missing categorical values were filled using the most frequent training category. Numerical features were standardized, while categorical features were one-hot encoded with unknown categories ignored.

The random evaluation used an 80:20 stratified split with random_state=42, producing 1,614 training records and 404 test records. The minority class was oversampled only in the training data, increasing the winner and non-winner training counts to the same size. The test set was left at its original class distribution. This final pipeline is the basis for the reported final random-split metrics.

4. Exploratory Data Analysis

The cleaned data shows a clear class imbalance: non-winners account for 73.29% of the candidate records, while winners account for 26.71%. This imbalance makes accuracy alone an incomplete measure of model quality. The notebook therefore reports precision, recall, F1 score and ROC-AUC alongside accuracy.

The notebook also examines missingness, the distribution of assets before and after log transformation, the relationship between numerical candidate characteristics and the winner label, and the difference in log-transformed assets between winners and non-winners. Before the final pipeline, the notebook's exploratory cleaning step found missing values in criminal cases (22 records), assets (26) and liabilities (23). The final model pipeline handles missingness separately within the training data.

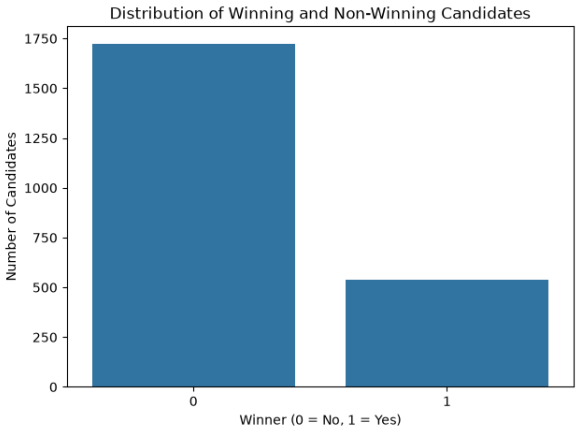


Fig. 1. Distribution of winning and non-winning candidates.

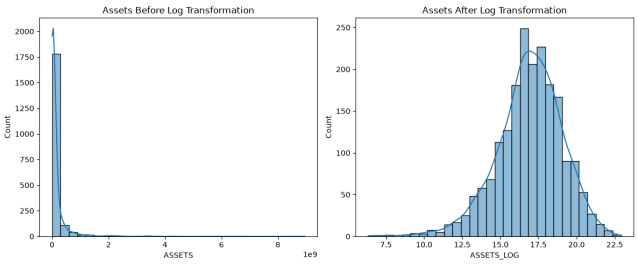


Fig. 2. Asset distribution before and after log transformation.

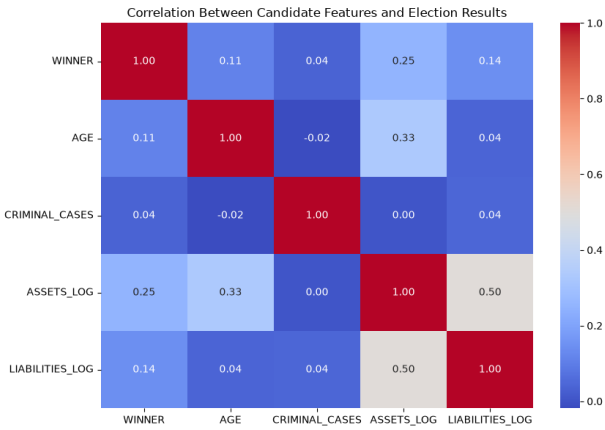


Fig. 3. Correlation among selected numerical variables.

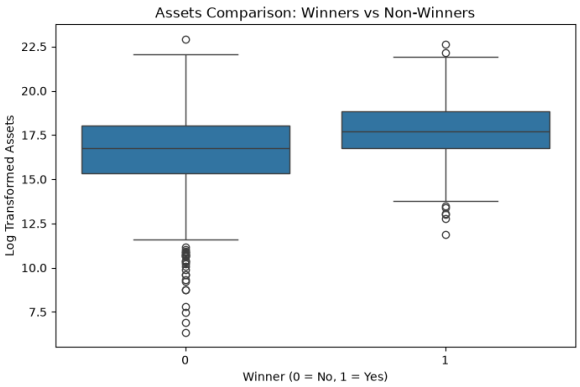


Fig. 4. Log-transformed assets by winner status.

5. Classification Models and Evaluation

Four supervised classification algorithms were implemented: Logistic Regression, Decision Tree, Gaussian Naive Bayes and k-Nearest Neighbors (KNN). Logistic Regression provides a linear classification boundary; Decision Tree learns a sequence of feature-based splits; Gaussian Naive Bayes applies a conditional-independence assumption; and KNN classifies a candidate using nearby training observations.

The Decision Tree used maximum depth 12, KNN used five neighbours, and Logistic Regression used a maximum of 1,000 iterations. The same prepared feature representation and balanced training set were used for the random-split model comparison. Performance was measured using accuracy, precision, recall, F1 score and ROC-AUC. F1 is useful here because it combines precision and recall for the minority winner class.

6. Results: Random Stratified Evaluation

The Decision Tree produced the highest F1 score (0.7751) and accuracy (0.8837). Logistic Regression achieved the highest ROC-AUC (0.8893), with an F1 score of 0.7456. KNN recorded an F1 score of 0.6426, while Gaussian Naive Bayes recorded 0.4644. Naive Bayes had high recall (0.9352), but its precision was low (0.3089), meaning that many candidates predicted as winners were actually non-winners.

Model	Acc.	Prec.	Rec.	F1	AUC
Decision Tree	0.8837	0.8020	0.7500	0.7751	0.8481
Logistic Reg.	0.8564	0.7083	0.7870	0.7456	0.8893
KNN	0.7550	0.5266	0.8241	0.6426	0.8404
Naive Bayes	0.4233	0.3089	0.9352	0.4644	0.5858

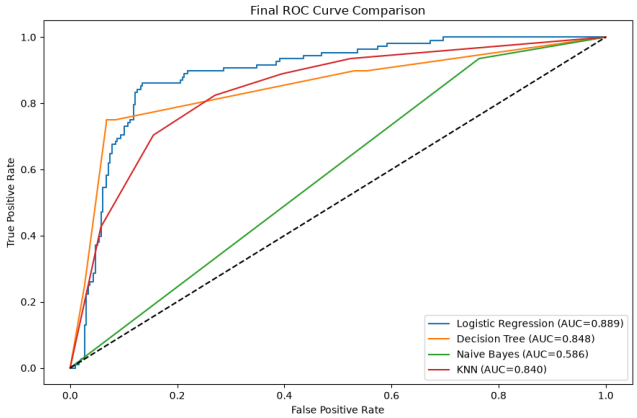


Fig. 5. ROC curve comparison from the notebook's final random-split models.

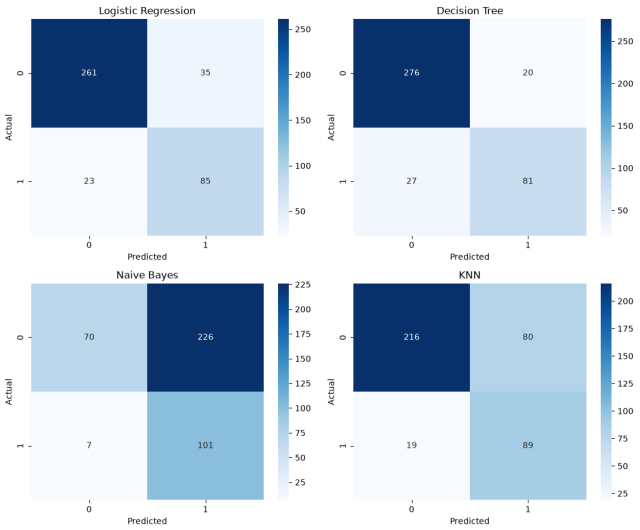


Fig. 6. Confusion matrices for the four classifiers.

7. Constituency-Based Validation

A random candidate-level split can place candidates from the same constituency in both training and test data. To examine performance under a stricter setting, the notebook also used GroupShuffleSplit with state and constituency combined as the grouping key. This kept complete constituencies out of the training set.

This grouped split contained 1,606 training candidates from 433 constituencies and 412 test candidates from 109 constituencies. Logistic Regression achieved the highest F1 score (0.7667) and ROC-AUC (0.9227). Decision Tree had the highest accuracy (0.8738) and precision (0.8132), but a lower recall of 0.6789. The difference in model rankings shows that the evaluation design affects which model appears strongest.

Model	Acc.	Prec.	Rec.	F1	AUC
Logistic Reg.	0.8641	0.7023	0.8440	0.7667	0.9227
Decision Tree	0.8738	0.8132	0.6789	0.7400	0.7659
KNN	0.7573	0.5290	0.7523	0.6212	0.8233
Naive Bayes	0.3835	0.2981	0.9817	0.4573	0.5798

8. Feature Interpretation

In the random-split Decision Tree, the BJP party indicator had the largest reported feature importance (0.3303). Log-transformed assets ranked next (0.0884), followed by the YSRCP party indicator (0.0632), Kerala state indicator (0.0558), DMK (0.0454) and AITC (0.0426). Criminal cases (0.0347) and age (0.0341) also appeared among the top features.

The grouped-training Decision Tree similarly ranked the BJP indicator first (0.3989), followed by log-transformed assets (0.1213), Kerala (0.0558) and DMK (0.0508). Feature importance reflects how the fitted tree used variables to make splits; it is not a measure of causal influence. Party and state indicators may also capture broader political and regional context not represented by the other candidate-level variables.

The grouped Logistic Regression odds-ratio output showed an odds ratio of 1.9456 for the standardized log-assets feature and 0.9127 for standardized log-liabilities. These values are conditional on the model, feature scaling and reference coding. They should not be interpreted as causal effects or as direct percentage changes in a candidate's real-world probability of winning.

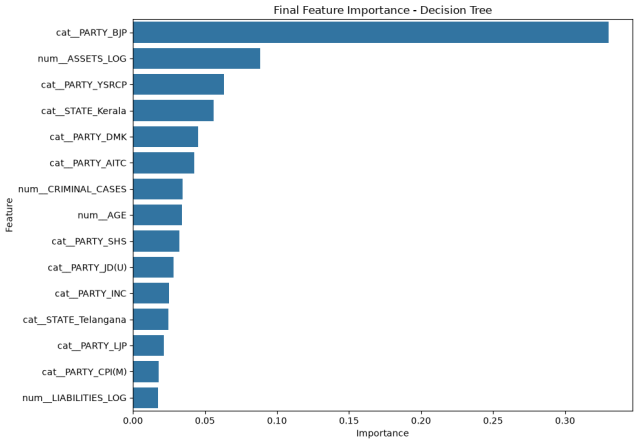


Fig. 7. Top Decision Tree feature importances in the final random-split model.

9. Discussion

The model comparisons suggest that the supplied candidate-level dataset contains information useful for classifying electoral outcomes. The Decision Tree performed best by F1 score in the random stratified evaluation, while Logistic Regression performed best by F1 and ROC-AUC when constituencies were held out. This difference is important: a model that performs well when candidates from the same constituency can appear in both sets may not behave in the same way when it is asked to generalize to constituencies it has not seen.

Party and state variables were prominent in the feature-importance results. This is plausible in an election setting, where a candidate's political party and regional context are closely connected with electoral competition. Assets also appeared as an important feature in both tree analyses. However, the results do not show that higher assets cause victory, nor do they show that criminal cases independently determine electoral outcomes.

10. Limitations and Ethical Considerations

The study is limited by the scope and quality of the supplied dataset. Affidavit-based fields may include self-reported information, and the notebook does not independently verify each declaration. Some records contain missing values, while the conversion of monetary strings extracts the first numeric amount. The results also depend on one random split and one grouped split; repeated constituency-grouped cross-validation would provide stronger evidence of stability.

Party and state indicators can dominate the model and may reflect political context rather than candidate merit. Predictions should not be used to label individual candidates as deserving or undeserving, and the model should not be treated as a tool for making political or legal judgments. The study is an academic exercise in classification and model evaluation, not a decision system for voters or election authorities.

11. Conclusion and Future Work

This project compared Logistic Regression, Decision Tree, Gaussian Naive Bayes and KNN for classifying candidate victory in the 2019 Indian General Election. Decision Tree achieved the highest F1 score in the random stratified evaluation (0.7751), while Logistic Regression achieved the highest ROC-AUC (0.8893). Under constituency-held-out validation, Logistic Regression achieved the strongest F1 score (0.7667) and ROC-AUC (0.9227).

Future work could repeat the grouped evaluation across several random seeds, use nested group-based validation for model selection, calibrate predicted probabilities, and compare performance across states and party groups. Additional verified candidate and constituency context may also improve the analysis. Any extension should continue to distinguish predictive association from causal explanation.

Declarations

Data availability. The analysis uses the supplied LS_2.0.csv file. The dataset listing is available at the Kaggle URL in Reference [1]. Check the dataset licence before redistribution.

Conflict of interest. [Add a declaration that accurately reflects your circumstances.]

Funding. [State whether the work received funding.]

Use of AI tools. [Disclose any AI assistance used for language editing, debugging or drafting, following university policy. Verify the paper and take responsibility for the submitted work.]

References

[1] P. Chauhan, “Indian Candidates for General Election 2019,” Kaggle dataset. Available: https://www.kaggle.com/datasets/prakrutchauhan/indian-candidates-for-general-election-2019

[2] MyNeta, candidate affidavit and election information. Available: https://www.myneta.info/

[3] Election Commission of India, election results and public election information. Available: https://www.eci.gov.in/

Appendix: Reproducibility

The supplied Jupyter Notebook contains the cleaning, exploratory analysis, preprocessing, model training, evaluation, feature-importance and constituency-held-out validation steps. Reported metrics were transcribed from the supplied notebook outputs.